

03_spatial_patterns

February 16, 2026

1 03 — Spatial patterns

Purpose: Visualize HH and LL clusters, examine distribution of HH frequency across seeds, and run connected-component analysis for hotspot stability. Uses one selected dataset.

1.1 Imports and path setup

```
[12]: import sys
from pathlib import Path

ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))
if str(ROOT / "src") not in sys.path:
    sys.path.insert(0, str(ROOT / "src"))

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from analysis.experiment_runner import _get_run_dirs
from analysis.preprocessing import get_transformed_test_features
from analysis.run_analysis import load_meta, load_P_test, load_split,
    run_spatial, select_rashomon_global, mean_variance
from analysis.spatial import extract_hh_components

RESULTS_DIR = ROOT / "results"
```

1.2 Select one dataset and compute HH masks across seeds

For each run we compute spatial analysis (Moran's I, LISA) and collect HH_mask, LL_mask, and pointwise variance.

```
[13]: DATASET = "compas"
K = 25
k_nn = 30

dataset_dir = RESULTS_DIR / DATASET
```

```

run_dirs = _get_run_dirs(dataset_dir)

spatial_by_run = []
for run_dir in run_dirs:
    n_cand = len(load_meta(run_dir))
    K_actual = min(K, n_cand)
    X_test = get_transformed_test_features(run_dir, DATASET)
    res = run_spatial(run_dir, X_test, K=K_actual, k=k_nn)
    res["run_dir"] = run_dir.name
    P_test = load_P_test(run_dir)
    idx = select_rashomon_global(run_dir, K=K_actual)
    res["mean_variance"] = mean_variance(P_test[idx], ddof=0)
    spatial_by_run.append(res)

print(f"Computed spatial metrics for {len(spatial_by_run)} runs.")

```

Computed spatial metrics for 10 runs.

1.3 Stability: HH count and variance summary per run

Plot number of HH points and mean pointwise variance per run to assess stability across seeds.

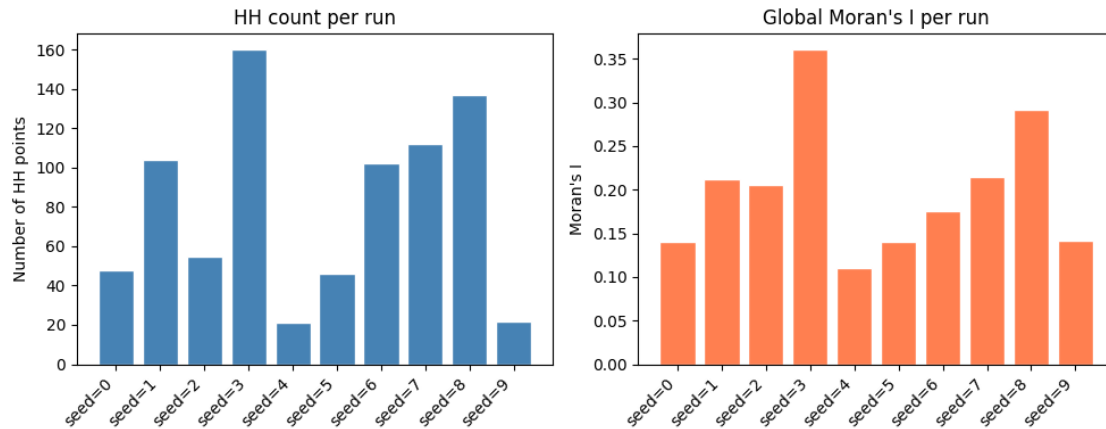
```

[14]: n_hh_per_run = [int(np.sum(r["HH_mask"])) for r in spatial_by_run]
run_labels = [r["run_dir"] for r in spatial_by_run]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
ax1.bar(range(len(n_hh_per_run)), n_hh_per_run, color="steelblue",
        edgecolor="white")
ax1.set_xticks(range(len(run_labels)))
ax1.set_xticklabels(run_labels, rotation=45, ha="right")
ax1.set_ylabel("Number of HH points")
ax1.set_title("HH count per run")

moran_per_run = [r["moran_i"] for r in spatial_by_run]
ax2.bar(range(len(moran_per_run)), moran_per_run, color="coral",
        edgecolor="white")
ax2.set_xticks(range(len(run_labels)))
ax2.set_xticklabels(run_labels, rotation=45, ha="right")
ax2.set_ylabel("Moran's I")
ax2.set_title("Global Moran's I per run")
plt.tight_layout()
plt.show()

```



1.4 Correlation: mean variance vs Moran's I per run

Plot mean variance (multiplicity) against global Moran's I (spatial clustering of variance) across runs.

- **Positive correlation:** More multiplicity → stronger spatial clustering of disagreement. Multiplicity concentrates in regions rather than being diffuse.
- **No (or weak) correlation:** Spatial structure of hotspots can exist even when overall multiplicity is small; the two are not simply coupled.

```
[15]: corr_df = pd.DataFrame({
    "run": [r["run_dir"] for r in spatial_by_run],
    "mean_variance": [r["mean_variance"] for r in spatial_by_run],
    "moran_i": [r["moran_i"] for r in spatial_by_run],
})
r_pearson = np.corrcoef(corr_df["mean_variance"], corr_df["moran_i"])[0, 1]

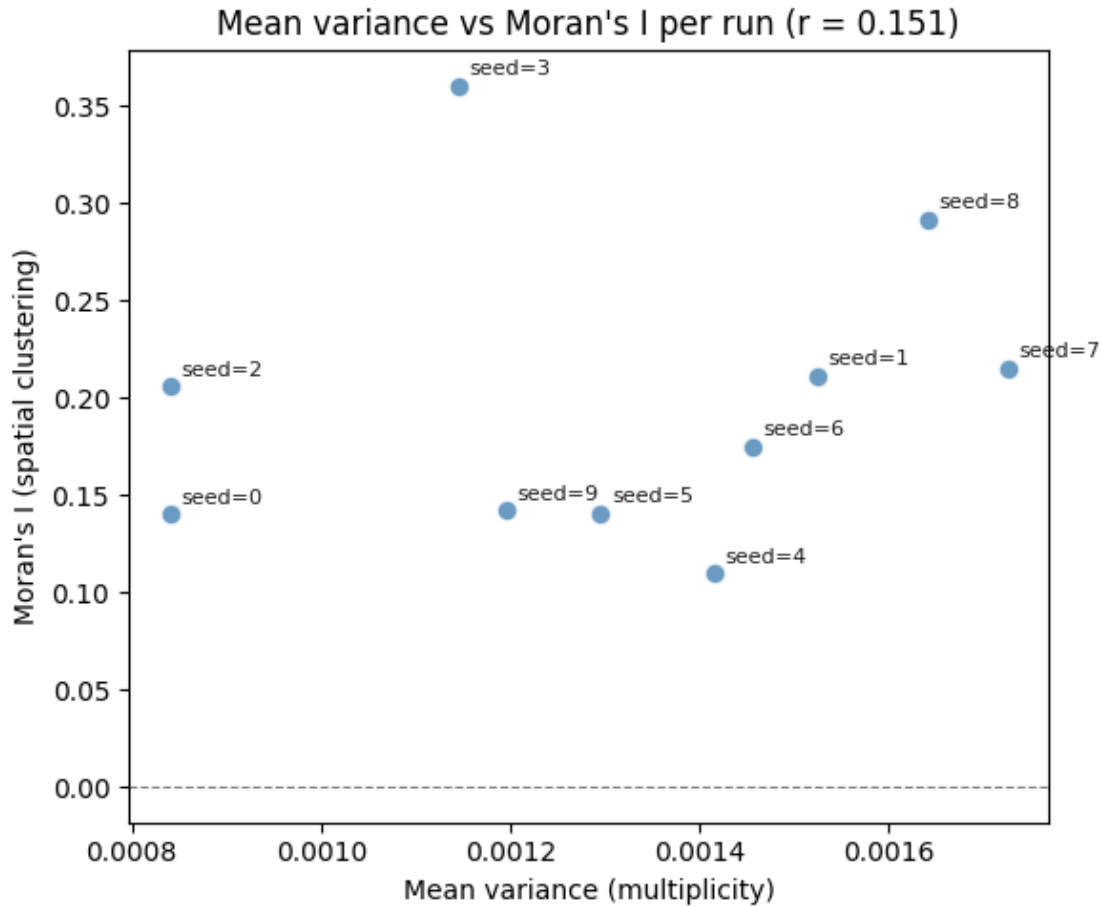
fig, ax = plt.subplots(figsize=(6, 5))
ax.scatter(corr_df["mean_variance"], corr_df["moran_i"], s=60, alpha=0.8,
           color="steelblue", edgecolor="white")
for _, row in corr_df.iterrows():
    ax.annotate(row["run"], (row["mean_variance"], row["moran_i"]), fontsize=8,
               alpha=0.9, xytext=(4, 4), textcoords="offset points")
ax.set_xlabel("Mean variance (multiplicity)")
ax.set_ylabel("Moran's I (spatial clustering)")
ax.set_title(f"Mean variance vs Moran's I per run (r = {r_pearson:.3f})")
ax.axhline(0, color="gray", linestyle="--", linewidth=0.8)
plt.tight_layout()
plt.show()

if r_pearson > 0.2:
```

```

print("→ Positive correlation: more multiplicity tends to coincide with_
↪ stronger spatial clustering.")
elif r_pearson < -0.2:
    print("→ Negative correlation: higher multiplicity tends to go with weaker_
↪ spatial clustering.")
else:
    print("→ Weak or no correlation: spatial structure of hotspots can exist_
↪ even when overall multiplicity is small.")

```



→ Weak or no correlation: spatial structure of hotspots can exist even when overall multiplicity is small.

1.5 Connected component analysis (hotspot regions)

For one run, extract HH connected components using the kNN graph and report region sizes. Uses `analysis.spatial.extract_hh_components`. The same `W` (from `spatial_analysis`) is used; kNN is directed. For undirected region connectivity you could symmetrize `W` before extraction.

```
[16]: # Use first run for component analysis
r0 = spatial_by_run[0]
HH_mask = r0["HH_mask"]
W_pysal = r0["W"]
W_sparse = W_pysal.to_sparse() if hasattr(W_pysal, "to_sparse") else W_pysal.
↳sparse

lisa_df = pd.DataFrame({
    "cluster": np.where(HH_mask, "HH", "NS"),
})
comp_id, components = extract_hh_components(lisa_df, W_sparse, min_size=5)

print(f"Run: {r0['run_dir']}")
print(f"HH points: {HH_mask.sum()}")
print(f"Connected components (min_size=5): {len(components)}")
for cid, inds in components.items():
    print(f"    Component {cid}: n = {len(inds)}")
```

```
Run: seed=0
HH points: 48
Connected components (min_size=5): 1
    Component 0: n = 45
```

1.6 Region persistence summary

Across runs, summarize number of HH components and largest component size (when available) to assess hotspot stability.

```
[17]: component_stats = []
for r in spatial_by_run:
    lisa_df = pd.DataFrame({"cluster": np.where(r["HH_mask"], "HH", "NS")})
    W = r["W"].to_sparse() if hasattr(r["W"], "to_sparse") else r["W"].sparse
    _, components = extract_hh_components(lisa_df, W, min_size=5)
    n_comp = len(components)
    max_size = max(len(inds) for inds in components.values()) if components
↳else 0
    component_stats.append({"run": r["run_dir"], "n_components": n_comp,
↳"max_component_size": max_size})

comp_df = pd.DataFrame(component_stats)
print("HH component summary (min_size=5):")
display(comp_df)
print(f"Mean n_components: {comp_df['n_components'].mean():.2f}")
print(f"Mean max_component_size: {comp_df['max_component_size'].mean():.2f}")
```

```
HH component summary (min_size=5):
```

	run	n_components	max_component_size
0	seed=0	1	45

1	seed=1	7	29
2	seed=2	3	35
3	seed=3	5	38
4	seed=4	2	10
5	seed=5	3	25
6	seed=6	6	33
7	seed=7	5	47
8	seed=8	3	92
9	seed=9	2	15

Mean n_components: 3.70

Mean max_component_size: 36.90